

# **UNIT IV Testing, Deployment and Maintenance**

## **Syllabus**

Software testing objectives, testing principles, verification vs validation, types of testing: Unit, Integration, System, Acceptance testing, Alpha and Beta testing, White-box testing, Black-box testing, test case design techniques, debugging, software maintenance (corrective, adaptive, perfective, preventive), software configuration management, risk management.

# Software testing objectives

Software testing is performed to ensure that a software system is reliable, efficient, and meets user expectations. The main objectives are:

## 1. Detect Defects (Bugs)

- Identify errors, faults, or missing requirements in the software.
- Ensure that issues are found early in the development process.

## 2. Ensure Quality

- Verify that the software meets specified requirements and standards.
- Maintain high product quality in terms of performance, usability, and reliability.

## 3. Verify and Validate Software

- **Verification:** Check whether the software is built correctly according to design.
- **Validation:** Check whether the correct software is built as per user needs.

#### **4. Improve Reliability**

- Ensure the system works consistently under different conditions.
- Reduce chances of system failures after deployment.

#### **5. Enhance Performance**

- Check speed, response time, and stability of the application.
- Ensure the system performs well under expected workload.

#### **6. Ensure Security**

- Identify vulnerabilities and protect against unauthorized access or data breaches.

#### **7. Provide Confidence to Stakeholders**

- Assure developers, clients, and users that the software works as intended.
- Build trust in the product before release.

## **8. Reduce Maintenance Cost**

- Early detection of defects reduces the cost of fixing issues later.
- Prevents expensive post-release fixes.

## **9. Ensure Compliance**

- Verify that the software meets legal, regulatory, and industry standards.

## **10. Improve User Satisfaction**

- Ensure the software is user-friendly and meets customer expectations.

# Software Testing Principles

## 1. Testing Shows Presence of Defects

- Testing can reveal bugs, but it **cannot prove that software is completely error-free**.
- It only reduces the probability of undiscovered defects.

### Example:

A mobile banking app is tested and no bugs are found during testing. However, after release, users discover a transaction failure issue.

👉 This shows testing found no defects earlier, but it **didn't guarantee a bug-free system**.

## 2. Exhaustive Testing is Impossible

- It is not feasible to test all possible inputs, paths, and conditions.
- Instead, testing focuses on **prioritization and risk-based testing**.

### Example:

A login form allows thousands of possible username-password combinations. Testing every combination is impossible.

👉 Testers check only **valid, invalid, and boundary cases** instead.

### 3. Early Testing (Shift Left)

- Testing should start as early as possible in the development lifecycle.
- Early detection reduces cost and effort.

#### Example:

If a requirement mistake is found during the design phase (e.g., wrong interest calculation in banking software), it is easy and cheap to fix.

👉 If found after deployment, it becomes **costly and complex**.

### 4. Defect Clustering

- A small number of modules contain most of the defects.
- Based on the **Pareto Principle (80/20 rule)**.

#### Example:

In an e-commerce website, most bugs are found in the **payment module**, while other modules have fewer issues.

👉 A few modules contain the majority of defects.

## 5. Pesticide Paradox

- Repeating the same tests over time becomes ineffective.
- Test cases must be **reviewed and updated regularly**.

### Example:

A tester repeatedly runs the same login test cases. Over time, no new bugs are found.

👉 New test cases (e.g., SQL injection, special characters) are needed to find new defects.

## 6. Testing is Context Dependent

- Testing approach varies depending on the application type.
- Example:
  - Banking software → needs high security testing
  - Gaming apps → focus on performance and usability

### Example:

Banking app → focus on **security & accuracy**

Gaming app → focus on **graphics & performance**

👉 Testing strategy depends on the application type.

## 7. Absence of Errors Fallacy

- Even if no defects are found, the software may still be unusable.
- Software must meet **user requirements**, not just be bug-free.

### Example:

An app has no bugs but has a confusing user interface. Users cannot navigate it properly.

👉 Even though it is bug-free, it **fails user expectations**.

## 8. Testing Requires Independence

- Testing is more effective when done by an independent team.
- Helps in finding unbiased defects.

### Example:

A developer may overlook their own coding mistakes.

An independent tester finds a critical bug in the same code.

👉 Independent testing improves defect detection.

## 9. Complete Testing is Not Possible

- Due to time and resource constraints, testing must be planned efficiently.
- Focus on critical functionalities.

### Example:

A large ERP system has thousands of features. Testing everything within deadlines is impossible.



Testers prioritize **critical features like finance and reporting.**

## 10. Testing is a Continuous Process

- Testing is performed throughout the software development lifecycle.
- Includes unit, integration, system, and acceptance testing.

### Example:

In Agile development, testing happens in every sprint:

Unit testing during coding

Integration testing after modules combine

System testing before release



Testing continues throughout development.

# Verification vs Validation Testing

These are two basic quality activities in software engineering.

## **Verification**

Verification checks whether the software is being developed **correctly according to specifications and design**.

It answers: **“Are we building the product right?”**

It is mostly a **static process**, which means the software does not need to be executed.

## **Examples of verification activities:**

- Requirement review
- Design review
- Code inspection
- Walkthroughs

## **Example:**

If the requirement says the login page must have username, password, and OTP fields, verification checks whether the design and code include all these fields correctly.

## Validation

Validation checks whether the developed software **actually satisfies user needs and expected use**.

It answers: **“Are we building the right product?”**

It is mostly a **dynamic process**, which means the software is executed and tested.

### **Examples of validation activities:**

- Functional testing
- System testing
- User acceptance testing
- Performance testing

### **Example:**

A login system may match the specification perfectly, but validation checks whether real users can log in easily, securely, and successfully in practical use.

# Types of software testing

## 1. Unit Testing

Testing individual components or functions of a program.

### **Example:**

Checking whether a function correctly calculates student grade percentage.

## 2. Integration Testing

Testing whether different modules work properly when combined.

### **Example:**

Checking whether the login module connects correctly with the database.

## 3. System Testing

Testing the complete software as a whole.

### **Example:**

Testing an entire online shopping website from login to payment.

#### **4. Acceptance Testing**

Testing whether the software meets user requirements and is ready for delivery.

**Example:**

A client checks whether the billing software works according to their business needs.

#### **5. Functional Testing**

Testing software functions against the specified requirements.

**Example:**

Checking whether the “Add to Cart” button adds products properly.

#### **6. Non-Functional Testing**

Testing performance, security, usability, reliability, etc.

**Example:**

Checking whether a website loads quickly when many users access it.

## **7. Manual Testing**

Testing done by humans without using automation tools.

### **Example:**

A tester manually checks each page of a website.

## **8. Automation Testing**

Testing done using scripts and tools.

### **Example:**

Using Selenium to test login functionality automatically.

## **9. Black Box Testing**

Testing without knowing the internal code structure.

### **Example:**

Tester checks input and output of a login form.

## **10. White Box Testing**

Testing with knowledge of internal code and logic.

### **Example:**

Checking all loops, conditions, and paths in the source code.

## **11. Grey Box Testing**

Testing with partial knowledge of internal structure.

### **Example:**

Tester knows the database design but tests through the user interface.

## **12. Regression Testing**

Testing whether old features still work after changes or bug fixes.

### **Example:**

After updating payment option, checking if login and cart still work properly.

## **13. Smoke Testing**

Basic testing to check whether the major functions of the software work.

### **Example:**

Checking whether the application installs, opens, and login works.

## **14. Sanity Testing**

Testing a specific function after making minor changes.

### **Example:**

After fixing the password reset bug, checking only the password reset feature.

## **15. Performance Testing**

Testing speed, stability, and responsiveness.

### **Example:**

Checking how fast a website responds when 1000 users use it at once.

## **16. Load Testing**

Testing system behavior under expected user load.

### **Example:**

Checking website performance with 500 simultaneous users.

## **17. Stress Testing**

Testing system beyond normal load limits.

### **Example:**

Checking what happens when 10,000 users access a website suddenly.

## **18. Security Testing**

Testing whether the system is protected from threats and attacks.

### **Example:**

Checking whether unauthorized users can access admin pages.

## **19. Usability Testing**

Testing how easy and user-friendly the system is.

### **Example:**

Checking whether users can easily place an order on an app.

## **20. Compatibility Testing**

Testing software on different devices, browsers, or operating systems.

### **Example:**

Checking whether a website works in Chrome, Firefox, and Edge.

## **21. Alpha Testing**

Testing done by developers or internal team before release.

### **Example:**

Company employees test the software in-house.

## **22. Beta Testing**

Testing done by real users before final release.

### **Example:**

A new mobile app is given to selected users for feedback.

# Software Maintenance

Software maintenance is the process of modifying software after delivery so it continues to work properly and remain useful.

There are **4 main types of software maintenance**:

## **1. Corrective Maintenance**

This is done to **fix errors or bugs** found after the software is released.

### **Purpose:**

To correct faults in design, coding, or implementation.

### **Examples:**

- Fixing login failure due to a coding bug
- Correcting wrong output in a billing system
- Removing crashes in a mobile app

## 2. Adaptive Maintenance

This is done to **make the software work in a changed environment.**

### **Purpose:**

To adapt the system to new operating conditions such as hardware, software, OS, business rules, or regulations.

### **Examples:**

- Updating software to run on a new version of Windows
- Modifying an app to support a new database
- Changing tax calculation rules in accounting software

### **3. Perfective Maintenance**

This is done to **improve the performance or add new features** based on user needs.

#### **Purpose:**

To enhance the system and make it more efficient, usable, or maintainable.

#### **Examples:**

- Adding a dark mode feature
- Improving system speed and response time
- Enhancing the user interface for better usability

### **4. Preventive Maintenance**

This is done to **prevent future problems** before they occur.

#### **Purpose:**

To improve software reliability and maintainability by reducing the chances of future faults.

#### **Examples:**

- Refactoring messy code
- Updating documentation
- Reorganizing database structure for future scalability
- Adding test cases to avoid future failures

# Risk management in software engineering

**Risk Management in Software Engineering** is a systematic process of identifying, analyzing, and controlling risks that may affect a software project's success (time, cost, quality, scope).

## ♦ 1. What is Risk?

A **risk** is any uncertain event or condition that can negatively impact a software project.

👉 Example:

- Requirement changes
- Budget overrun
- Technology failure
- Team skill gaps

## ♦ 2. Types of Risks

### 1. Technical Risks

- Use of new or untested technology
- Integration issues
- Performance problems

## **2. Project Risks**

- Poor planning
- Schedule delays
- Resource shortages

## **3. Business Risks**

- Market changes
- Customer dissatisfaction
- Financial loss

## **4. Operational Risks**

- Process failures
- Lack of communication
- Infrastructure issues

## **◆ 3. Risk Management Process**

### **Step 1: Risk Identification**

Identify possible risks early.

- Brainstorming
- Expert judgment
- Past project analysis

👉 Output: List of risks

## Step 2: Risk Analysis

Evaluate risk probability and impact.

- **Qualitative Analysis** (High, Medium, Low)
- **Quantitative Analysis** (Numerical values)

👉 Example:

- Probability = 0.7
- Impact = High

## Step 3: Risk Prioritization

Rank risks based on severity.

👉 Formula:

**Risk Exposure = Probability × Impact**

## Step 4: Risk Mitigation (Planning)

Develop strategies to reduce risks.

Common strategies:

- **Avoidance** → Change plan to eliminate risk
- **Mitigation** → Reduce impact/probability
- **Transfer** → Outsource or insure
- **Acceptance** → Accept risk with backup plan

## Step 5: Risk Monitoring & Control

Track risks throughout the project.

- Regular reviews
- Risk status updates
- Corrective actions

### ◆ 4. Risk Management Techniques

- Risk Register
- SWOT Analysis
- Fishbone Diagram - a visual tool used to identify and analyze the root causes of a problem
- Checklists

### ◆ 5. Risk Register (Example)

| Risk               | Probability | Impact | Mitigation               |
|--------------------|-------------|--------|--------------------------|
| Requirement change | High        | High   | Frequent client meetings |
| Developer leaves   | Medium      | High   | Knowledge sharing        |
| Server failure     | Low         | High   | Backup systems           |

## ◆ 6. Importance of Risk Management

- Improves project success rate
- Reduces uncertainty
- Saves cost and time
- Enhances decision-making
- Ensures better quality software

# Software Configuration Management (SCM)

**Software Configuration Management (SCM)** in Software Engineering is the process of systematically controlling, organizing, and tracking changes made to software throughout its lifecycle.

## ◆ 1. What is SCM?

SCM ensures that all software components (code, documents, requirements) are:

- Properly versioned
- Controlled
- Consistent
- Traceable

👉 In simple terms:

**SCM manages “what changed, who changed it, and why it was changed.”**

## ◆ 2. Objectives of SCM

- Maintain consistency of software products
- Control changes efficiently
- Enable team collaboration
- Reduce errors and conflicts
- Support rollback to previous versions

### ◆ 3. Key Components of SCM

#### 1. Configuration Identification

- Identify all items to be managed (called Configuration Items - CIs)
- Examples: source code, design documents, test cases

#### 2. Version Control

- Manage different versions of software
- Track history of changes

👉 Tools:

- Git
- Subversion

#### 3. Change Control

- Process of approving/rejecting changes
- Managed by Change Control Board (CCB)

Steps:

1. Submit change request
2. Analyze impact
3. Approve/reject
4. Implement change

#### **4. Configuration Status Accounting**

- Records and reports status of changes
- Tracks:
  - What changes were made
  - Who made them
  - Current version

#### **5. Configuration Auditing**

- Ensures product matches requirements
- Verifies changes are correctly implemented

#### **◆ 4. SCM Process**

1. Identify configuration items
2. Control changes
3. Record and track changes
4. Audit configurations

## ◆ 5. SCM Tools

Common tools used in industry:

- GitHub
- GitLab
- Bitbucket

## ◆ 6. Importance of SCM

- Prevents version conflicts
- Supports teamwork in large projects
- Enables easy debugging and rollback
- Improves software quality
- Ensures project stability

## ◆ 7. Example

👉 Without SCM:

- Multiple developers overwrite code
- No record of changes

👉 With SCM:

- Each change is tracked
- Versions can be restored
- Collaboration becomes smooth

## 8. Key Differences (SCM vs Risk Management)

| Aspect | SCM                  | Risk Management        |
|--------|----------------------|------------------------|
| Focus  | Managing changes     | Managing uncertainties |
| Goal   | Maintain consistency | Reduce project risks   |
| Nature | Technical process    | Management process     |